

Trabalho Prático: Gerador de Horário Escolar

1. Introdução

Este trabalho tem como objetivo a conceção e implementação de um sistema automático para geração de horários escolares semanais. O problema é formulado como um **Problema de Satisfação de Restrições (CSP)** e otimização inteira, utilizando a biblioteca **Google OR-Tools (CP-SAT)**.

Para a manipulação dos dados de entrada, optou-se por utilizar o módulo nativo **CSV** da biblioteca standard de Python (em vez de **pandas**), garantindo maior leveza, ausência de dependências externas e uma conversão direta para as estruturas nativas (listas e dicionários) que o solver manipula com facilidade.

O sistema responde aos seguintes eixos principais:

1. Respeito integral por todos os requisitos operacionais (**R1 a R8**).
2. Minimização dos tempos mortos ("buracos") nos horários dos professores (**O1**).
3. Capacidade de adaptação incremental estável face a alterações de recursos (**R9**).

```
In [ ]: import marimo as mo

from ortools.sat.python import cp_model
```

2. Leitura e Preparação dos Dados (R8)

Os dados de entrada são lidos diretamente dos ficheiros CSV (**turmas.csv**, **disciplinas.csv**, **salas.csv** e **disponibilidade_excecoes.csv**), garantindo a total ausência de dados fixos (*hardcoded*) no código.

A leitura é dividida em duas funções:

- **_ler_csv_com_validacao**: Uma função utilitária e genérica que abre qualquer CSV com **csv.DictReader**, valida se os cabeçalhos obrigatórios existem e devolve o texto limpo.
- **carregar_dados**: Orquestra o carregamento dos quatro ficheiros e converte os valores em tipos nativos de Python (inteiros para a carga horária e capacidades, booleanos para duplos períodos, etc.).

A semana letiva é composta por 5 dias (**Seg** a **Sex**), com 5 tempos letivos diários (períodos 1 a 5).

```

In [ ]: import csv
        from pathlib import Path

# Nota: a leitura e validação dos CSV foi desenvolvida com apoio de um
# assistente de IA e revista pelos autores.
def _ler_csv_com_validacao(
    caminho_ficheiro: Path, colunas_obrigatorias: set[str]
) -> list[dict[str, str]]:
    """Abre um CSV, valida se as colunas obrigatórias existem no cabeçalho
    e devolve uma lista de dicionários com strings sem espaços nos extremos
    (strip).
    """
    if not caminho_ficheiro.exists():
        raise FileNotFoundError(
            f"Ficheiro obrigatório não encontrado: '{caminho_ficheiro}'"
        )

    linhas_limpas = []
    with open(caminho_ficheiro, mode="r", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        cabecalhos = set(col.strip() for col in (reader.fieldnames or []))

        # Validação do cabeçalho
        if not colunas_obrigatorias.issubset(cabecalhos):
            em_falta = colunas_obrigatorias - cabecalhos
            raise ValueError(
                f"Ficheiro '{caminho_ficheiro.name}' inválido. "
                f"Colunas em falta: {sorted(em_falta)}. Cabeçalho "
                f"encontrado: {reader.fieldnames}"
            )

        for linha in reader:
            linhas_limpas.append({
                k.strip(): (v.strip() if v is not None else "")
                for k, v in linha.items()
                if k is not None
            })

    return linhas_limpas

def carregar_dados(diretoria_dados: str | Path = "dados") -> dict:
    """Lê e valida os 4 ficheiros CSV da diretoria indicada (R8)
    e adiciona a estrutura temporal da semana escolar (5 dias x 5
    períodos).
    """
    pasta = Path(diretoria_dados)
    if not pasta.exists():
        raise FileNotFoundError(f"A diretoria '{pasta}' não foi
        encontrada.")

    # 1. Calendário escolar (5 dias x 5 períodos conforme o enunciado)

```

```

dias = ["Seg", "Ter", "Qua", "Qui", "Sex"]
periodos = [1, 2, 3, 4, 5]

# 2. turmas.csv
linhas_turmas = _ler_csv_com_validacao(
    pasta / "turmas.csv",
    colunas_obrigatorias={"turma"},
)
turmas = [l["turma"] for l in linhas_turmas if l["turma"]]

# 3. salas.csv
linhas_salas = _ler_csv_com_validacao(
    pasta / "salas.csv",
    colunas_obrigatorias={"sala", "tipo", "quantidade"},
)
salas = [
    {
        "sala": l["sala"],
        "tipo": l["tipo"].lower(),
        "quantidade": int(l["quantidade"]),
    }
    for l in linhas_salas
]

# 4. disciplinas.csv
linhas_disc = _ler_csv_com_validacao(
    pasta / "disciplinas.csv",
    colunas_obrigatorias={
        "disciplina",
        "professor",
        "carga_semanal",
        "duplo_perodo",
        "sala_especial",
    },
)
disciplinas = [
    {
        "disciplina": l["disciplina"],
        "professor": l["professor"],
        "carga_semanal": int(l["carga_semanal"]),
        "duplo_perodo": l["duplo_perodo"].lower() in ("sim", "true",
"1"),
        "sala_especial": l["sala_especial"] or None,
    }
    for l in linhas_disc
]

# 5. disponibilidade_excecoes.csv
linhas_disp = _ler_csv_com_validacao(
    pasta / "disponibilidade_excecoes.csv",
    colunas_obrigatorias={"professor", "dia", "periodo"},
)
indisponibilidades = {
    (l["professor"], l["dia"], int(l["periodo"]))
    for l in linhas_disp
    if l["dia"] in dias and int(l["periodo"]) in periodos
}

```

```

    }

    return {
        "dias": dias,
        "periodos": periodos,
        "turmas": turmas,
        "salas": salas,
        "disciplinas": disciplinas,
        "indisponibilidades": indisponibilidades,
    }

dados_h0 = carregar_dados("dados/")
dados_h1 = carregar_dados("dados_v2/")
dados_h3 = carregar_dados("dados_v3/")

```

3. Modelo CP-SAT

Variáveis de decisão

Para cada turma t , disciplina d , dia dia e período p existe uma variável booleana:

$$x_{t,d,dia,p} \in \{0, 1\}$$

que vale 1 se a turma t tem aula da disciplina d nesse dia e nesse período, e 0 caso contrário. Como cada disciplina tem um único professor, a variável determina também o professor que dá a aula.

```

In [ ]: def criar_variaveis(model, dados):
    turmas = dados["turmas"]
    disciplinas = [_d["disciplina"] for _d in dados["disciplinas"]]
    dias = dados["dias"]
    periodos = dados["periodos"]

    # Criar as variáveis de decisão booleanas x[turma, disciplina, dia,
    periodo]
    x = {}
    for _t in turmas:
        for _d in disciplinas:
            for _dia in dias:
                for _p in periodos:
                    x[_t, _d, _dia, _p] =
model.NewBoolVar(f"x_{_t}_{_d}_{_dia}_{_p}")
    return x

```

R1: uma turma não pode ter duas aulas em simultâneo

Para cada turma, dia e período, a soma das variáveis de todas as disciplinas é no máximo 1:

$$\forall t, dia, p: \sum_d x_{t,d,dia,p} \leq 1$$

```
In [ ]: def restricao_r1(model, x, dados):
    turmas = dados["turmas"]
    disciplinas = [_d["disciplina"] for _d in dados["disciplinas"]]
    dias = dados["dias"]
    periodos = dados["periodos"]

    # R1: Uma turma não pode ter duas aulas em simultâneo (no máx. 1 por tempo)
    for _t in turmas:
        for _dia in dias:
            for _p in periodos:
                model.Add(sum(x[_t, _d, _dia, _p] for _d in disciplinas) <=
1)
```

R2: cada disciplina cumpre exatamente a carga semanal

Para cada turma e disciplina, o número total de tempos atribuídos na semana é igual à carga semanal c_d lida de `disciplinas.csv`:

$$\forall t, d: \sum_{dia} \sum_p x_{t,d,dia,p} = c_d$$

```
In [ ]: def restricao_r2(model, x, dados):
    turmas = dados["turmas"]
    dias = dados["dias"]
    periodos = dados["periodos"]

    #R2: Cumprir exatamente a carga semanal de cada disciplina por turma
    for d_info in dados["disciplinas"]:
        _d_nome = d_info["disciplina"]
        carga = d_info["carga_semanal"]

        for _t in turmas:
            # A soma de todos os tempos da disciplina 'd_nome' na semana
            # para a turma 't' tem de ser igual à carga
            model.Add(
                sum(x[_t, _d_nome, _dia, _p] for _dia in dias for _p in
periodos) == carga
            )
```

R3 e R4: uma aula por dia e blocos de duplo período

As duas restrições ficam juntas porque partilham as variáveis auxiliares dos blocos.

Disciplinas normais (R3): no máximo uma aula por dia, por turma:

$$\forall t, d, dia: \sum_p x_{t,d,dia,p} \leq 1$$

Disciplinas com duplo_periodo=sim (R3 e R4): para cada turma, disciplina e dia cria-se uma variável booleana $b_{t,d,dia,p}$, com $p \in \{1, 2, 3, 4\}$, que vale 1 se um bloco de 2 tempos começa no período p (ocupando p e $p + 1$).

No máximo um bloco por dia (R3):

$$\sum_{p=1}^4 b_{t,d,dia,p} \leq 1$$

Um tempo só tem aula se pertencer a um bloco (R4), ou seja, se o bloco começou nesse período ou no anterior:

$$x_{t,d,dia,p} = b_{t,d,dia,p-1} + b_{t,d,dia,p} \quad \text{com } b_{t,d,dia,0} = b_{t,d,dia,5} = 0$$

Assim nunca existe um tempo isolado: a disciplina tem 0 ou 2 tempos consecutivos em cada dia.

```
In [ ]: def restricao_r3_r4(model, x, dados):
    turmas = dados["turmas"]
    dias = dados["dias"]
    periodos = dados["periodos"]

    # R3 e R4: aulas por dia e blocos duplos
    for _d in dados["disciplinas"]:
        _d_nome = _d["disciplina"]
        e_duplo = _d["duplo_periodo"]

        for _t in turmas:
            for _dia in dias:

                if not e_duplo:
                    # R3 para disciplinas normais: no máximo 1 aula por dia
                    model.Add(
                        sum(x[(_t, _d_nome, _dia, _p)] for _p in periodos)
                        <= 1
                    )

                else:
                    # R3 e R4 para disciplinas com duplo_periodo=sim:
                    # Nota: a R3 é da autoria do grupo; a formulação da R4
                    # (blocos de 2 tempos)
                    # foi desenvolvida com apoio de um assistente de IA e
                    # revista pelos autores.
                    # Criar variáveis booleanas para o início do bloco
                    # duplo (períodos 1 a 4)
                    bloco_inicio = {
                        _p:
                        model.NewBoolVar(f"bloco_{_t}_{_d_nome}_{_dia}_{_p}")
                        for _p in [1, 2, 3, 4]
                    }

                    # R3: No máximo 1 bloco duplo por dia
                    model.Add(sum(bloco_inicio.values()) <= 1)
```

```

# R4: Ligar as variáveis x normais ao início do bloco
model.Add(x[_t, _d_nome, _dia, 1] == bloco_inicio[1])
model.Add(
    x[_t, _d_nome, _dia, 2] == bloco_inicio[1] +
bloco_inicio[2]
)
model.Add(
    x[_t, _d_nome, _dia, 3] == bloco_inicio[2] +
bloco_inicio[3]
)
model.Add(
    x[_t, _d_nome, _dia, 4] == bloco_inicio[3] +
bloco_inicio[4]
)
model.Add(x[_t, _d_nome, _dia, 5] == bloco_inicio[4])

```

R5 e R6: professores

As duas restrições ficam juntas porque partilham o dicionário `disciplinas_prof`, que associa cada professor ao conjunto D_{prof} das disciplinas que leciona.

R5: um professor não pode dar duas aulas em simultâneo. Em cada dia e período, a soma das aulas de todas as suas disciplinas, em todas as turmas, é no máximo 1:

$$\forall prof, dia, p : \sum_t \sum_{d \in D_{prof}} x_{t,d,dia,p} \leq 1$$

R6: um professor só dá aulas quando está disponível. Sendo I o conjunto de indisponibilidades lido de `disponibilidade_excecoes.csv`, as variáveis correspondentes são forçadas a 0:

$$\forall (prof, dia, p) \in I, \forall t, \forall d \in D_{prof} : x_{t,d,dia,p} = 0$$

```

In [ ]: def restricao_r5_r6(model, x, dados):
    turmas = dados["turmas"]
    dias = dados["dias"]
    periodos = dados["periodos"]

    #R5 Um professor não pode dar duas aulas em simultâneo, mesmo que sejam
    a turmas ou disciplinas diferentes.
    professores = set(_d["professor"] for _d in dados["disciplinas"])
    disciplinas_prof = {_prof: [] for _prof in professores}

    for _d in dados["disciplinas"]:
        disciplinas_prof[_d["professor"]].append(_d["disciplina"])

    for _prof in professores:
        disc_prof = disciplinas_prof[_prof]
        for _dia in dias:
            for _p in periodos:
                model.Add(

```

```

sum(
    x[_t, _d, _dia, _p]
    for _t in turmas
    for _d in disc_prof
) <= 1
)

# R6: Um professor só pode dar aulas nos tempos em que está disponível
for _prof, _dia, _p in dados["indisponibilidades"]:
    # Se o professor lecionar disciplinas no sistema
    if _prof in disciplinas_prof:
        for _d in disciplinas_prof[_prof]:
            for _t in turmas:
                # Força a variável booleana a ser 0 naquele período
                model.Add(x[_t, _d, _dia, _p] == 0)

```

R7: capacidade das salas

As disciplinas são separadas pelo tipo de sala que ocupam: D_s é o conjunto das disciplinas que usam a sala especial s e D_{normal} o das restantes. Em cada dia e período, o número de aulas a decorrer em cada tipo de sala não pode exceder a quantidade q definida em `salas.csv`.

Para cada sala especial s :

$$\forall dia, p : \sum_t \sum_{d \in D_s} x_{t,d,dia,p} \leq q_s$$

Para as salas normais:

$$\forall dia, p : \sum_t \sum_{d \in D_{normal}} x_{t,d,dia,p} \leq q_{normal}$$

Limitação. O modelo conta as salas por tipo e não atribui uma sala concreta a cada aula. Por isso, na construção incremental só se medem mudanças de tempo, e não mudanças de sala.

```

In [ ]: def restricao_r7(model, x, dados):
    turmas = dados["turmas"]
    dias = dados["dias"]
    periodos = dados["periodos"]

    # R7: Capacidade de Salas (Especiais e Normais)
    discs_normais=[]
    discs_especiais_sala= {}
    for _d in dados["disciplinas"]:
        sala_esp = _d.get("sala_especial")
        _d_nome = _d["disciplina"]

        if sala_esp and sala_esp.strip() != "":
            if sala_esp not in discs_especiais_sala:
                discs_especiais_sala[sala_esp] = []

```

```

        discs_especiais_sala[sala_esp].append(_d_nome)
    else: discs_normais.append(_d_nome)

cap_salas = {s["sala"]: s["quantidade"] for s in dados["salas"]}
qtd_salas_normais = sum(
    s["quantidade"] for s in dados["salas"] if s["tipo"] == "normal"
)

for _dia in dias:
    for _p in periodos:
        # A) Capacidade para cada sala especial
        for sala_esp, discs_esp in discs_especiais_sala.items():
            cap_esp = cap_salas.get(sala_esp)
            model.Add(
                sum(
                    x[_t, _d, _dia, _p]
                    for _t in turmas
                    for _d in discs_esp
                ) <= cap_esp
            )

        # B) Capacidade para salas normais
        model.Add(
            sum(
                x[_t, _d, _dia, _p]
                for _t in turmas
                for _d in discs_normais
            ) <= qtd_salas_normais
        )

```

O1: minimizar os buracos nos horários dos professores

Um buraco é um tempo livre de um professor, no meio do dia, entre a sua primeira e a sua última aula desse dia.

Ocupação do professor. Num dado dia e período, o professor está ocupado se alguma turma tem aula de uma das suas disciplinas. Pela R5 esta soma vale 0 ou 1:

$$o_{prof,dia,p} = \sum_t \sum_{d \in D_{prof}} x_{t,d,dia,p}$$

Variáveis auxiliares. Para cada professor, dia e período criam-se três booleanas:

- $a_{prof,dia,p}$ (**antes**): o professor já deu alguma aula nesse dia, no período p ou antes;
- $d_{prof,dia,p}$ (**depois**): o professor ainda dá alguma aula nesse dia, no período p ou depois;
- $b_{prof,dia,p}$ (**buraco**): o período p é um buraco.

Restrições. Omitindo os índices $prof$ e dia :

$$a_p \geq o_p \quad a_p \geq a_{p-1}$$

$$d_p \geq o_p \quad d_p \geq d_{p+1}$$

$$b_p \geq a_p + d_p - 1 - o_p$$

Um período é buraco quando há aulas antes, há aulas depois e o professor está livre.
Exemplo com aulas nos períodos 1, 2 e 4:

período	1	2	3	4	5
ocupado	1	1	0	1	0
antes	1	1	1	1	1
depois	1	1	1	1	0
antes + depois - 1 - ocupado	0	0	1	0	0

O único buraco é o período 3. O período 5 também está livre, mas `depois` já é 0, por isso não conta.

Objetivo. A função devolve a soma de todos os buracos, que o `resolver` minimiza:

$$\min \sum_{prof} \sum_{dia} \sum_p b_{prof,dia,p}$$

As restrições são todas do tipo $\geq$: só obrigam as variáveis a subir para 1 quando é preciso. Quem as faz descer é o objetivo, porque ao minimizar a soma o solver nunca marca um buraco que não exista.

```
In [ ]: def total_buracos(model, x, dados):
    # Nota: esta função foi desenvolvida com apoio de um assistente de IA e
    revista pelos autores.
    turmas = dados["turmas"]
    dias = dados["dias"]
    periodos = dados["periodos"]

    # Dicionário professor -> lista das disciplinas que leciona (como na
    R5)
    disciplinas_prof = {}
    for d in dados["disciplinas"]:
        if d["professor"] not in disciplinas_prof:
            disciplinas_prof[d["professor"]] = []
        disciplinas_prof[d["professor"]].append(d["disciplina"])

    buracos = [] # todas as variáveis "buraco" criadas, para somar no fim

    for prof in disciplinas_prof:
        for dia in dias:
            antes = {}
            depois = {}
            for p in periodos:
                antes[p] = model.NewBoolVar(f"antes_{prof}_{dia}_{p}")
                depois[p] = model.NewBoolVar(f"depois_{prof}_{dia}_{p}")
```

```

        for i in range(len(periodos)):
            p = periodos[i]

            # ocupado = 1 se o professor dá aula neste tempo, a
            qualquer turma
            ocupado = sum(
                x[t, d_nome, dia, p]
                for t in turmas
                for d_nome in disciplinas_prof[prof]
            )

            # antes: fica a 1 se há aula agora, ou se já estava a 1 no
            período anterior
            model.Add(antes[p] >= ocupado)
            if i > 0:
                model.Add(antes[p] >= antes[periodos[i - 1]])

            # depois: fica a 1 se há aula agora, ou se está a 1 no
            período seguinte
            model.Add(depois[p] >= ocupado)
            if i < len(periodos) - 1:
                model.Add(depois[p] >= depois[periodos[i + 1]])

            # buraco: há aulas antes, há aulas depois e o professor
            está livre
            buraco = model.NewBoolVar(f"buraco_{prof}_{dia}_{p}")
            model.Add(buraco >= antes[p] + depois[p] - 1 - ocupado)
            buracos.append(buraco)

    return sum(buracos)

```

Construção do modelo

A função `construir_modelo` cria um modelo novo, as variáveis de decisão e aplica as restrições R1 a R7 pela ordem acima.

```

In [ ]: def construir_modelo(dados):
    """Constrói o modelo CP-SAT com as restrições R1 a R7 para os `dados`
    indicados e devolve o par (model, x), onde x[turma, disciplina, dia,
    periodo] são as variáveis de decisão booleanas.
    """

    # 1. Iniciar o modelo CP-SAT
    model = cp_model.CpModel()

    # 2. Criar as variáveis de decisão booleanas x[turma, disciplina, dia,
    periodo]
    x = criar_variaveis(model, dados)

    # 3. Adicionar as restrições R1 a R7
    restricao_r1(model, x, dados)
    restricao_r2(model, x, dados)
    restricao_r3_r4(model, x, dados)

```

```

restricao_r5_r6(model, x, dados)
restricao_r7(model, x, dados)

return model, x

```

4. Resolução e construção incremental (R9)

A função `resolver(dados, base=None)` constrói o modelo, resolve-o e devolve o horário como um conjunto de tuplos `(turma, disciplina, dia, periodo)`, juntamente com o tempo de resolução.

Sem horário base (`base=None`): o problema é resolvido do zero, com as restrições R1 a R7 e o objetivo O1 (minimizar os buracos dos professores, com `total_buracos`).

Com horário base (`base=H0`): a resolução é incremental e reaproveita o horário anterior de duas formas:

- **Hint:** para as aulas que já existiam em H_0 , sugere-se ao solver o valor que cada variável tinha (`AddHint`), para a procura começar perto do horário anterior.
- **Objetivo:** minimiza-se o número de aulas de H_0 que deixam de estar no mesmo tempo. Cada parcela $(1 - x)$ vale 0 se a aula fica onde estava e 1 se muda:

$$\min \sum_{(t,d,dia,p) \in H_0} (1 - x_{t,d,dia,p})$$

Só entram na soma as aulas de H_0 que ainda existem nos dados novos, para a mesma função servir quando há turmas ou disciplinas novas.

Optou-se por hint e objetivo em vez de fixar as aulas não afetadas, que é a outra técnica sugerida no enunciado, porque fixar pode tornar o problema impossível quando a alteração obriga a mexer numa aula que parecia não afetada. Com o objetivo, todas as aulas podem mudar se for preciso, mas cada mudança tem um custo.

Um modelo só pode ter um objetivo, por isso cada caminho tem o seu: O1 na resolução do zero e o número de aulas alteradas na incremental. O enunciado não exige que H_1 seja ótimo em relação a O1.

O solver tem um limite de 30 segundos. Se não encontrar solução (modelo inviável ou tempo esgotado), a função devolve `None` no lugar do horário.

```

In [ ]: def resolver(dados, base=None):
        # Nota: esta função foi desenvolvida com apoio de um assistente de IA e
        # revista pelos autores.
        # 1. Modelo novo: cada chamada tem o seu modelo e as suas variáveis

        model, x = construir_modelo(dados)

```

```

# 3. Parte incremental: só existe quando é dado um horário base
if base is not None:
    # Pares (turma, disciplina) que já existiam no horário base
    pares_base = set()
    for t, d_nome, dia, p in base:
        pares_base.add((t, d_nome))

# 3a. Hint: sugerir ao solver os valores que as variáveis tinham no
base
for chave in x:
    t, d_nome, dia, p = chave
    if (t, d_nome) in pares_base:
        if chave in base:
            model.AddHint(x[chave], 1)
        else:
            model.AddHint(x[chave], 0)

# 3b. Objetivo: (1 - x) vale 0 se a aula fica no seu tempo e 1 se
muda
mudancas = []
for chave in base:
    if chave in x:                # só conta se a aula ainda existir nos
dados novos
        mudancas.append(1 - x[chave])
model.Minimize(sum(mudancas))

# 3c. Resolução do zero (sem base): o objetivo é o 01, minimizar os
buracos
else:
    model.Minimize(total_buracos(model, x, dados))

# 4. Resolver (com limite de tempo)
solver = cp_model.CpSolver()
solver.parameters.max_time_in_seconds = 30
status = solver.Solve(model)
tempo = solver.WallTime()

# 5. Sem solução: devolve None no lugar do horário
if status != cp_model.OPTIMAL and status != cp_model.FEASIBLE:
    return None, tempo

# 6. Passar a solução para um set de tuplos
horario = set()
for chave in x:
    if solver.Value(x[chave]) == 1:
        horario.add(chave)
return horario, tempo

```

5. Apresentação do horário

A função `mostrar_horario(dados, horario, titulo)` imprime uma grelha por turma, com os dias nas colunas e os períodos nas linhas. Cada célula mostra a disciplina e o professor, ou `---` se a turma não tem aula nesse tempo.

Se o horário for `None`, é mostrada uma mensagem de erro em vez da grelha.

```
In [ ]: def mostrar_horario(dados, horario, titulo):
    # Nota: esta função foi desenvolvida com apoio de um assistente de IA e
    revista pelos autores.
    turmas = dados["turmas"]
    disciplinas = [_d["disciplina"] for _d in dados["disciplinas"]]
    dias = dados["dias"]
    periodos = dados["periodos"]

    # 1. Mapeamento auxiliar de disciplina -> professor a partir dos dados
    prof_por_disciplina = {
        _d["disciplina"]: _d["professor"] for _d in dados["disciplinas"]
    }

    # 2. Tratar o resultado da resolução
    if horario is not None:
        print(f"=== {titulo}: solução encontrada com sucesso! ===\n")

        # Largura de cada coluna da grelha para garantir alinhamento
        perfeito
        col_w = 34

        for _t in turmas:
            # Cabeçalho dos dias da semana
            cabecalho = f"{'Tempo':^12}|" + "".join(
                f"{'_d':^{{col_w}}}" for _d in dias
            )
            separador = "-" * len(cabecalho)

            print("=" * len(cabecalho))
            print(f" HORÁRIO SEMANAL {titulo}: TURMA {_t}")
            print("=" * len(cabecalho))
            print(cabecalho)
            print(separador)

            # Linhas correspondentes a cada período letivo
            for _p in periodos:
                linha_conteudo = [f"{'_p':^12} Período: ^12}|" ]

                for _dia in dias:
                    aula_str = "---"
                    for _d_nome in disciplinas:
                        # Verifica se a aula está atribuída a este tempo
                        if (_t, _d_nome, _dia, _p) in horario:
                            _prof = prof_por_disciplina.get(_d_nome, "Prof.
                            ?")

                            aula_str = f"{{_d_nome}} ({{_prof}})"
                            break

                    linha_conteudo.append(f"{{aula_str: ^{{col_w}}}}|")

                print("".join(linha_conteudo))

            print(separador)
```

```

        print("\n")

    else:
        print(f"ERRO: {titulo} sem solução (modelo INVIÁVEL ou tempo
esgotado).")

```

Contagem de buracos

A função `contar_buracos(dados, horario)` conta os buracos diretamente num horário já resolvido, sem usar o solver: para cada professor e dia, procura os períodos livres entre a primeira e a última aula. Serve para verificar o valor de O1 de forma independente do modelo.

```

In [ ]: def contar_buracos(dados, horario):
        # Nota: esta função foi desenvolvida com apoio de um assistente de IA e
        revista pelos autores.
        if horario is None:
            return None

        # Dicionário disciplina -> professor
        prof_por_disciplina = {}
        for d in dados["disciplinas"]:
            prof_por_disciplina[d["disciplina"]] = d["professor"]

        # Lista dos professores, sem repetidos
        professores = []
        for d in dados["disciplinas"]:
            if d["professor"] not in professores:
                professores.append(d["professor"])

        total = 0
        for prof in professores:
            for dia in dados["dias"]:
                # Períodos em que este professor dá aula neste dia
                ocupados = []
                for t, d_nome, dia_aula, p in horario:
                    if dia_aula == dia and prof_por_disciplina.get(d_nome) ==
prof:
                        ocupados.append(p)

                if len(ocupados) > 0:
                    primeira = min(ocupados)
                    ultima = max(ocupados)
                    # Buraco: período livre entre a primeira e a última aula
                    for p in dados["periodos"]:
                        if p > primeira and p < ultima and p not in ocupados:
                            total = total + 1

        return total

```

Verificação automática das restrições

A função `verificar_horario(dados, horario)` confirma, diretamente sobre um horário já resolvido e sem usar o solver, que as restrições R1 a R7 são respeitadas, e devolve a lista das que falharam (vazia se estiver tudo certo). A R8 fica de fora porque diz respeito à leitura dos dados a partir dos ficheiros CSV, e não ao horário gerado.

```
In [ ]: def verificar_horario(dados, horario):
    if horario is None:
        return ["sem horário"]

    erros = [] # mensagens das restrições que falharam

    # R1: uma turma não pode ter duas aulas em simultâneo
    for t in dados["turmas"]:
        for dia in dados["dias"]:
            for p in dados["periodos"]:
                total = 0
                for turma, disciplina, dia_aula, p_aula in horario:
                    if turma == t and dia_aula == dia and p_aula == p:
                        total = total + 1
                if total > 1:
                    erros.append(f"R1: {t} tem {total} aulas em simultâneo em {dia} {p}")

    # R2: cada turma tem exatamente a carga semanal de cada disciplina
    for t in dados["turmas"]:
        for d in dados["disciplinas"]:
            total = 0
            for turma, disciplina, dia, p in horario:
                if turma == t and disciplina == d["disciplina"]:
                    total = total + 1
            if total != d["carga_semanal"]:
                erros.append(f"R2: {t} tem {total} tempos de {d['disciplina']}")

    # R3 e R4: no máximo uma aula por dia, ou um bloco de 2 tempos seguidos
    for t in dados["turmas"]:
        for d in dados["disciplinas"]:
            for dia in dados["dias"]:
                # Períodos em que esta turma tem esta disciplina neste dia
                tempos = []
                for turma, disciplina, dia_aula, p_aula in horario:
                    if turma == t and disciplina == d["disciplina"] and
dia_aula == dia:
                        tempos.append(p_aula)
                tempos.sort()
                total = len(tempos)

                if d["duplo_periodo"]:
                    # R4: ou não há aula nesse dia, ou há exatamente 2
tempos seguidos
                    bloco_certo = total == 2 and tempos[1] == tempos[0] + 1
                    if total > 0 and not bloco_certo:
                        erros.append(f"R4: {t} tem {d['disciplina']} nos
tempos {tempos} de {dia}")
```

```

        else:
            # R3: no máximo uma aula por dia
            if total > 1:
                erros.append(f"R3: {t} tem {total} aulas de
{d['disciplina']} em {dia}")

            # Dicionário disciplina -> professor (usado na R5 e na R6)
            prof_por_disciplina = {}
            for d in dados["disciplinas"]:
                prof_por_disciplina[d["disciplina"]] = d["professor"]

            #R5: Um professor não pode ter duas aulas no mesmo tempo
            professores = set(prof_por_disciplina.values())
            for prof in professores:
                for dia in dados["dias"]:
                    for p in dados["periodos"]:
                        total = 0
                        for turma, disciplina, dia_aula, p_aula in horario:
                            if prof_por_disciplina[disciplina] == prof and dia_aula
== dia and p_aula == p:
                                total = total + 1
                            if total>1:
                                erros.append(f"R5: {prof} tem {total} aulas em
simultâneo em {dia} {p}")

            #R6: Um professor não pode dar aula num tempo em que está indisponível.
            for turma, disciplina, dia, p in horario:
                prof = prof_por_disciplina[disciplina]
                if (prof, dia, p) in dados["indisponibilidades"]:
                    erros.append(f"R6: {prof} dá aula a {turma} em {dia} {p}, mas
está indisponível")

            # R7: em cada tempo, as aulas em cada tipo de sala não excedem a
quantidade
            # Nota: a verificação da R7 foi desenvolvida com apoio de um assistente
de IA e revista pelos autores.
            # Quantidade de salas normais (soma de todas as de tipo "normal")
            # e quantidade de cada sala especial, pelo nome
            qtd_normais = 0
            qtd_especiais = {}
            for s in dados["salas"]:
                if s["tipo"] == "normal":
                    qtd_normais = qtd_normais + s["quantidade"]
                else:
                    qtd_especiais[s["sala"]] = qtd_especiais.get(s["sala"], 0) +
s["quantidade"]

            # Sala especial de cada disciplina (None se usar uma sala normal)
            sala_por_disciplina = {}
            for d in dados["disciplinas"]:
                sala_por_disciplina[d["disciplina"]] = d["sala_especial"]

            # Lista das salas especiais pedidas pelas disciplinas, sem repetidos
            salas_especiais = []
            for d in dados["disciplinas"]:
                if d["sala_especial"] and d["sala_especial"] not in

```

```

salas_especiais:
    salas_especiais.append(d["sala_especial"])

    for dia in dados["dias"]:
        for p in dados["periodos"]:
            # R7 (salas normais): aulas de disciplinas sem sala especial
            neste tempo
                total = 0
                for turma, disciplina, dia_aula, p_aula in horario:
                    if dia_aula == dia and p_aula == p and not
sala_por_disciplina[disciplina]:
                        total = total + 1
                    if total > qtd_normais:
                        erros.append(f"R7: {total} aulas em salas normais em {dia}
{p}, mas só há {qtd_normais}")

                # R7 (salas especiais): aulas que precisam de cada sala
                especial neste tempo
                    for sala in salas_especiais:
                        total = 0
                        for turma, disciplina, dia_aula, p_aula in horario:
                            if dia_aula == dia and p_aula == p and
sala_por_disciplina[disciplina] == sala:
                                total = total + 1
                                # se a sala não estiver em salas.csv, a quantidade é 0
                                if total > qtd_especiais.get(sala, 0):
                                    erros.append(f"R7: {total} aulas em {sala} em {dia}
{p}, mas só há {qtd_especiais.get(sala, 0)}")

    return erros

```

6. Testes

H0: horário inicial

Horário gerado do zero a partir dos dados iniciais (`dados/`). Serve de base para as construções incrementais seguintes.

```

In [ ]: h0, t_h0 = resolver(dados_h0)
        mostrar_horario(dados_h0, h0, "H0")

```

=== H0: solução encontrada com sucesso! ===

HORÁRIO SEMANAL H0: TURMA 7ºA

Tempo	Seg	Ter
Qua	Qui	
Sex		
1º Período	Matemática (Prof. Ana)	Português (Prof. Bruno)
2º Período	Inglês (Prof. Diana)	Ciências (Prof. Carla)
3º Período	Português (Prof. Bruno)	Inglês (Prof. Diana)
4º Período	Educação Física (Prof. Eduardo)	História (Prof. Diana)
5º Período	Educação Física (Prof. Eduardo)	Matemática (Prof. Ana)

HORÁRIO SEMANAL H0: TURMA 7ºB

Tempo	Seg	Ter
Qua	Qui	
Sex		
1º Período	Ciências (Prof. Carla)	Ciências (Prof. Carla)
2º Período	Matemática (Prof. Ana)	Inglês (Prof. Diana)
3º Período	História (Prof. Diana)	

=== H1 do zero: solução encontrada com sucesso! ===

HORÁRIO SEMANAL H1 do zero: TURMA 7ºA

Tempo	Seg	Ter
Qua	Qui	
Sex		

1º Período	Matemática (Prof. Ana)	---
Matemática (Prof. Ana)	Ciências (Prof. Carla)	
Ciências (Prof. Carla)		
2º Período	Inglês (Prof. Diana)	Português (Prof. Bruno)
Português (Prof. Bruno)	---	

3º Período	História (Prof. Diana)	Inglês (Prof. Diana)
---		---
Matemática (Prof. Ana)		
4º Período	---	História (Prof. Diana)
Ciências (Prof. Carla)	Matemática (Prof. Ana)	Educ
ação Física (Prof. Eduardo)		
5º Período	Português (Prof. Bruno)	---
---	Português (Prof. Bruno)	Educ
ação Física (Prof. Eduardo)		

HORÁRIO SEMANAL H1 do zero: TURMA 7ºB

Tempo	Seg	Ter
Qua	Qui	
Sex		

1º Período	---	Português (Prof. Bruno)
Português (Prof. Bruno)	---	

2º Período	Matemática (Prof. Ana)	História (Prof. Diana)
Matemática (Prof. Ana)	---	

3º Período	Ciências (Prof. Carla)	Ciências (Prof. Carla)
Ciências (Prof. Carla)	---	

```

---
|
4º Período | História (Prof. Diana) | Matemática (Prof. Ana)
| Educação Física (Prof. Eduardo) | Português (Prof. Bruno) |
Português (Prof. Bruno) |
5º Período | Inglês (Prof. Diana) | Inglês (Prof. Diana)
| Educação Física (Prof. Eduardo) | Matemática (Prof. Ana) |
---
|
-----
-----
-----

```

H1 incremental

O mesmo problema de `dados_v2/`, mas resolvido a partir de H_0 (`base=h0`): o solver recebe o horário anterior como hint e minimiza o número de aulas que mudam.

```

In [ ]: h1_inc, t_inc = resolver(dados_h1, base=h0)
        mostrar_horario(dados_h1, h1_inc, "H1 incremental")

```

=== H1 incremental: solução encontrada com sucesso! ===

HORÁRIO SEMANAL H1 incremental: TURMA 7ºA

Tempo	Seg	Ter
Qua	Qui	
Sex		
1º Período	Matemática (Prof. Ana)	Português (Prof. Bruno)
2º Período	Inglês (Prof. Diana)	Ciências (Prof. Carla)
3º Período	Português (Prof. Bruno)	Inglês (Prof. Diana)
4º Período	Educação Física (Prof. Eduardo)	História (Prof. Diana)
5º Período	Educação Física (Prof. Eduardo)	Matemática (Prof. Ana)

HORÁRIO SEMANAL H1 incremental: TURMA 7ºB

Tempo	Seg	Ter
Qua	Qui	
Sex		
1º Período	Ciências (Prof. Carla)	Ciências (Prof. Carla)
2º Período	Matemática (Prof. Ana)	Inglês (Prof. Diana)
3º Período	História (Prof. Diana)	

=== v3 do zero: solução encontrada com sucesso! ===

HORÁRIO SEMANAL v3 do zero: TURMA 7ºA

Tempo	Seg	Ter
Qua	Qui	
Sex		
1º Período	---	---
Geografia (Prof. Fábio)	---	
Matemática (Prof. Ana)		
2º Período	Português (Prof. Bruno)	História (Prof. Diana)
Ciências (Prof. Carla)	Ciências (Prof. Carla)	

3º Período	Ciências (Prof. Carla)	Matemática (Prof. Ana)
Matemática (Prof. Ana)	Geografia (Prof. Fábio)	
Português (Prof. Bruno)		
4º Período	História (Prof. Diana)	Português (Prof. Bruno)
Educação Física (Prof. Eduardo)	Português (Prof. Bruno)	

5º Período	Matemática (Prof. Ana)	---
Educação Física (Prof. Eduardo)	Inglês (Prof. Gabriela)	
Inglês (Prof. Gabriela)		

HORÁRIO SEMANAL v3 do zero: TURMA 7ºB

Tempo	Seg	Ter
Qua	Qui	
Sex		
1º Período	---	---
Matemática (Prof. Ana)	Matemática (Prof. Ana)	
Português (Prof. Bruno)		
2º Período	---	Ciências (Prof. Carla)
Geografia (Prof. Fábio)	Geografia (Prof. Fábio)	
Matemática (Prof. Ana)		
3º Período	Português (Prof. Bruno)	Português (Prof. Bruno)
História (Prof. Diana)	Português (Prof. Bruno)	

Ciências (Prof. Carla)				
4º Período		Matemática (Prof. Ana)		---
		Ciências (Prof. Carla)		Educação Física (Prof. Eduardo)
Inglês (Prof. Gabriela)				
5º Período		---		Inglês (Prof. Gabriela)
		---		Educação Física (Prof. Eduardo)
História (Prof. Diana)				

=====

=====

=====

HORÁRIO SEMANAL v3 do zero: TURMA 7ºC

=====

=====

=====

Tempo		Seg		Qui	Ter	
		Qua				
Sex						

1º Período		Português (Prof. Bruno)		---		
		---		---		

2º Período		Inglês (Prof. Gabriela)		---		
		Matemática (Prof. Ana)		Matemática (Prof. Ana)		
Português (Prof. Bruno)						
3º Período		Matemática (Prof. Ana)		Ciências (Prof. Carla)		
		Ciências (Prof. Carla)		---		
Geografia (Prof. Fábio)						
4º Período		Ciências (Prof. Carla)		Matemática (Prof. Ana)		
		História (Prof. Diana)		Geografia (Prof. Fábio)		
Educação Física (Prof. Eduardo)						
5º Período		História (Prof. Diana)		Português (Prof. Bruno)		
		Inglês (Prof. Gabriela)		Português (Prof. Bruno)		
Educação Física (Prof. Eduardo)						

Cenário v3 incremental

O mesmo cenário de `dados_v3/`, resolvido a partir de H_0 . As aulas da turma nova e da disciplina nova não existem em H_0 , por isso não entram no objetivo nem recebem hint: o solver é livre de as colocar onde couberem.

```
In [ ]: h3_inc, t3_inc = resolver(dados_h3, base=h0)
        mostrar_horario(dados_h3, h3_inc, "v3 incremental")
```

=== v3 incremental: solução encontrada com sucesso! ===

HORÁRIO SEMANAL v3 incremental: TURMA 7ºA

Tempo	Seg	Ter
Qua	Qui	
Sex		
1º Período	Matemática (Prof. Ana)	Português (Prof. Bruno)
Português (Prof. Bruno)		
2º Período	Inglês (Prof. Gabriela)	Ciências (Prof. Carla)
Geografia (Prof. Fábio)	Geografia (Prof. Fábio)	
3º Período	Português (Prof. Bruno)	Inglês (Prof. Gabriela)
	Português (Prof. Bruno)	
4º Período	Educação Física (Prof. Eduardo)	História (Prof. Diana)
Ciências (Prof. Carla)	Ciências (Prof. Carla)	
História (Prof. Diana)		
5º Período	Educação Física (Prof. Eduardo)	Matemática (Prof. Ana)
Matemática (Prof. Ana)		

HORÁRIO SEMANAL v3 incremental: TURMA 7ºB

Tempo	Seg	Ter
Qua	Qui	
Sex		
1º Período	Ciências (Prof. Carla)	Ciências (Prof. Carla)
Ciências (Prof. Carla)	Matemática (Prof. Ana)	
2º Período	Matemática (Prof. Ana)	Inglês (Prof. Gabriela)
	Português (Prof. Bruno)	
3º Período	História (Prof. Diana)	Português (Prof. Bruno)

```

História (Prof. Diana) |
4º Período | Português (Prof. Bruno) | Educação Física (Prof. Edua
rdo) | Matemática (Prof. Ana) | Inglês (Prof. Gabriela)
| Matemática (Prof. Ana) |
5º Período | Geografia (Prof. Fábio) | Educação Física (Prof. Edua
rdo) | --- | Geografia (Prof. Fábio)
| Português (Prof. Bruno) |
-----
-----
-----

```

```

=====
=====
=====
HORÁRIO SEMANAL v3 incremental: TURMA 7ºC
=====
=====
=====
Tempo | Seg | Ter |
| Qua | Qui |
Sex |
-----
-----
1º Período | Português (Prof. Bruno) | ---
| --- | Inglês (Prof. Gabriela) |
Geografia (Prof. Fábio) |
2º Período | --- | Português (Prof. Bruno)
| --- | --- |
Matemática (Prof. Ana) |
3º Período | Ciências (Prof. Carla) | Ciências (Prof. Carla)
| Matemática (Prof. Ana) | Matemática (Prof. Ana) |
Português (Prof. Bruno) |
4º Período | Matemática (Prof. Ana) | ---
| Inglês (Prof. Gabriela) | Geografia (Prof. Fábio) | Educ
ação Física (Prof. Eduardo) |
5º Período | História (Prof. Diana) | História (Prof. Diana)
| Ciências (Prof. Carla) | Português (Prof. Bruno) | Educ
ação Física (Prof. Eduardo) |
-----
-----
-----

```

Verificação dos horários gerados

A função `verificar_horario` é aplicada aos cinco horários gerados acima. Cada linha mostra `OK` se o horário respeita R1 a R7, ou a lista das restrições violadas.

```

In [ ]: for _nome, _dados, _horario in [
        ("H0", dados_h0, h0),
        ("H1 do zero", dados_h1, h1_zero),
        ("H1 incremental", dados_h1, h1_inc),

```

```

    ("v3 do zero", dados_h3, h3_zero),
    ("v3 incremental", dados_h3, h3_inc),
]:
    _erros = verificar_horario(_dados, _horario)
    print(f"{_nome}: {'OK' if not _erros else _erros}")

```

H0: OK

H1 do zero: OK

H1 incremental: OK

v3 do zero: OK

v3 incremental: OK

7. Comparação: do zero vs. incremental

Para cada horário mostram-se três medidas (o H_0 aparece como referência, com 0 aulas alteradas por definição):

- **Tempo mediano:** o tempo de uma única resolução varia de execução para execução, por isso a função `medir` resolve o mesmo problema 5 vezes e devolve a mediana dos tempos.
- **Aulas alteradas:** as aulas de H_0 que não estão no mesmo tempo no horário novo:

$$\text{aulas alteradas} = |H_0 \setminus H_1|$$

- **Buracos:** o número de buracos nos horários dos professores (O1), contado com `contar_buracos`. Os horários resolvidos do zero minimizam O1; os incrementais minimizam as aulas alteradas, por isso podem ter mais buracos.

```

In [ ]: def medir(dados, base=None, repeticoes=5):
        tempos = []
        for i in range(repeticoes):
            horario, tempo = resolver(dados, base=base)
            tempos.append(tempo)
        return sorted(tempos)[len(tempos) // 2]    # mediana

```

```

In [ ]: _tabela = []
        for _nome, _dados, _base, _horario in [
            ("H0", dados_h0, None, h0),
            ("H1 do zero", dados_h1, None, h1_zero),
            ("H1 incremental", dados_h1, h0, h1_inc),
            ("v3 do zero", dados_h3, None, h3_zero),
            ("v3 incremental", dados_h3, h0, h3_inc),
        ]:
            _tabela.append({
                "Horário": _nome,
                "Tempo mediano (s)": round(medir(_dados, base=_base), 4),
                "Aulas alteradas": len(h0 - _horario),
                "Buracos": contar_buracos(_dados, _horario),
            })
        mo.ui.table(_tabela, selection=None)

```

Horário	Tempo mediano (s)	Aulas alteradas	Buracos	
H0	0,0340	0	0	
H1 do zero	0,0319	28	0	
H1 incremental	0,0117	2	0	
v3 do zero	0,0492	33	0	
v3 incremental	0,0179	2	12	

5 rows, 4 columns

No selection

Aulas que mudaram na construção incremental

Para `dados_v2/` mostram-se os tempos de onde as aulas saíram e para onde foram.

```
In [ ]: print("v2: saíram de", sorted(h0 - h1_inc), "e foram para", sorted(h1_inc - h0))
```

v2: saíram de [('7ºA', 'Matemática', 'Sex', 5), ('7ºB', 'Matemática', 'Sex', 4)] e foram para [('7ºA', 'Matemática', 'Sex', 3), ('7ºB', 'Matemática', 'Sex', 2)]

8. Conclusão

A tabela da secção 7 permite comparar as duas formas de obter um horário novo depois de uma alteração aos recursos. Os valores abaixo são os obtidos nas execuções feitas durante o desenvolvimento.

- **Aulas alteradas.** É a diferença mais clara. Com `dados_v2/`, resolver do zero muda 28 aulas em relação a H_0 , enquanto a construção incremental muda 2, que são as aulas de Matemática da sexta-feira nos tempos em que a Prof. Ana deixou de estar disponível. Com `dados_v3/`, do zero mudam 33 aulas e na incremental 2.
- **Tempo.** Nas medições feitas, a mediana do tempo da construção incremental foi inferior à da resolução do zero nos dois cenários. Todos os tempos ficam abaixo de meio segundo e variam de execução para execução, por isso com dados deste tamanho a diferença de tempo é uma evidência mais fraca do que o número de aulas alteradas.
- **Buracos.** É o preço da construção incremental. Os horários resolvidos do zero minimizam O1 e ficam com 0 buracos. Os incrementais minimizam as aulas alteradas e não olham aos buracos: com `dados_v2/` ficam com 0 ou 1, e com `dados_v3/` chegam a 12, porque as aulas da turma nova são colocadas nos tempos que sobram. O enunciado admite esta troca, ao não exigir que H_1 seja ótimo em relação a O1.

Todos os horários gerados passam na verificação automática de R1 a R7.

9. Utilização de ferramentas de IA

No desenvolvimento deste trabalho recorreu-se ao apoio de modelos de linguagem (LLM), usados como ferramenta de consulta e de revisão: para esclarecer dúvidas sobre o CP-SAT e o Marimo, depurar erros, organizar o código em funções e rever a redação das explicações. As decisões de modelação e a validação dos resultados foram verificadas pelos autores.

Por transparência, ficam as conversas que serviram de apoio:

- Claude (Anthropic): <https://claude.ai/share/3e88341a-10b5-4ac2-b97d-821a4856df24>
- Gemini (Google): <https://share.gemini.google/MuGTbAdONH2P>